K-means

From textbook - please study textbook

In this article, we'll describe the **k-means algorithm** and provide practical examples using **R** software. We will use the demo data sets "USArrests".

1. Specify the number of clusters (K) to be created (by the analyst)

```
data("USArrests")      # Loading the data set
df <- scale(USArrests) # Scaling the data
```

View the firt 3 rows of the data

```
head(df, n = 3)
```

```
##             Murder   Assault   UrbanPop         Rape
## Alabama 1.24256408 0.7828393 -0.5209066 -0.003416473
## Alaska  0.50786248 1.1068225 -1.2117642  2.484202941
## Arizona 0.07163341 1.4788032  0.9989801  1.042878388
```

K-means function in R and arguments

kmeans(x, centers, iter.max = 10, nstart = 1)

- **x**: numeric matrix, numeric data frame or a numeric vector

```
## install.packages("factoextra")

library(factoextra)
```

```
## Warning: package 'factoextra' was built under R version 4.0.5
```

```
## Loading required package: ggplot2
```

```
## Warning: package 'ggplot2' was built under R version 4.0.5
```

```
## Welcome! Want to learn more? See two factoextra-related books at https://goo.gl/ve3WBa
```

Estimating the optimal number of clusters

The k-means clustering requires the users to specify the number of clusters to be generated.

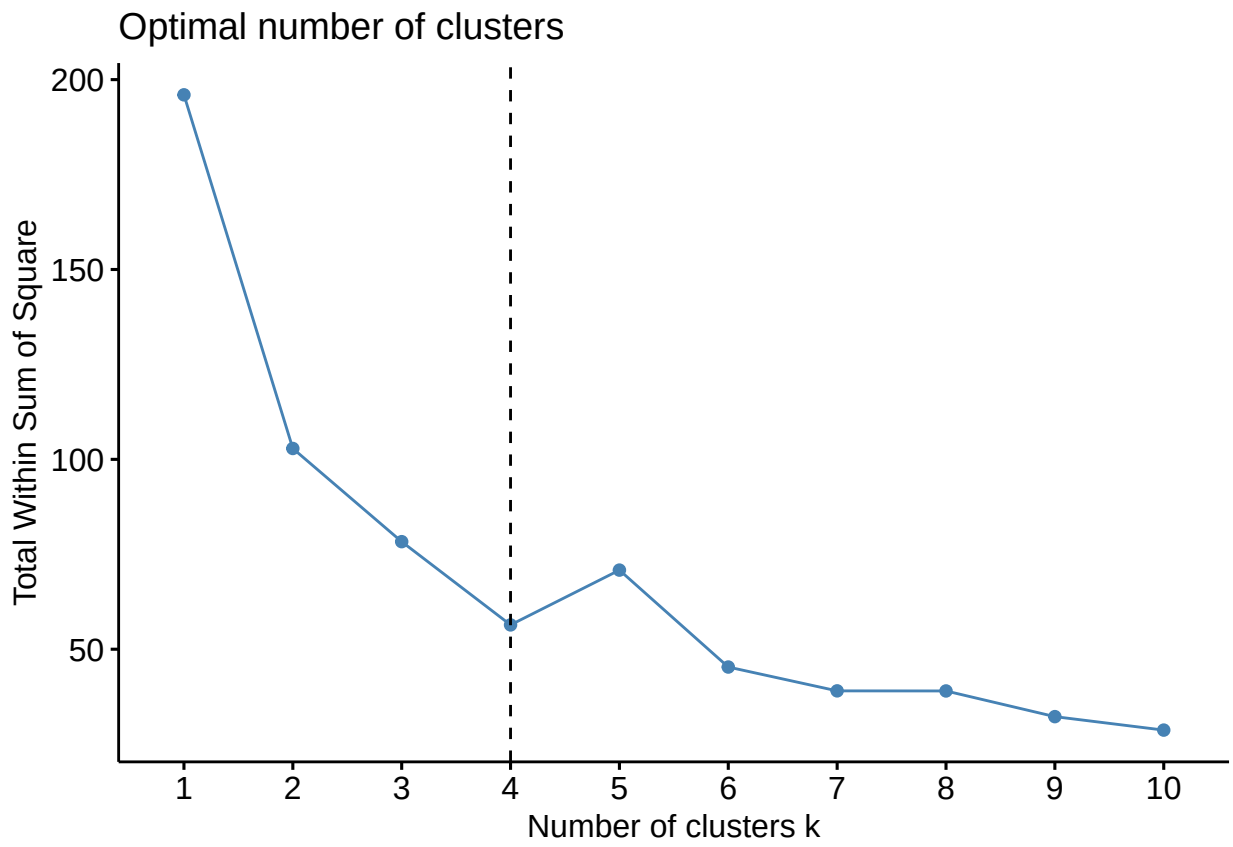
One fundamental question is: How to choose the right number of expected clusters (k)?

Different methods will be presented in the chapter "cluster evaluation and validation statistics".

Here, we provide a simple solution. The idea is to compute k-means clustering using different values of clusters k. Next, the wss (within sum of square) is drawn according to the number of clusters. The location of a bend (knee) in the plot is generally considered as an indicator of the appropriate number of clusters.

The R function *fviz_nbclust()* [in *factoextra* package] provides a convenient solution to estimate the optimal number of clusters.

```
fviz_nbclust(df, kmeans, method = "wss") +
    geom_vline(xintercept = 4, linetype = 2)
```



The plot above represents the variance within the clusters. It decreases as k increases, but it can be seen a bend (or "elbow") at k = 4. This bend indicates that additional clusters beyond the fourth have little value. In the next section, we'll classify the observations into 4 clusters.

Compute k-means clustering with k = 4

```
set.seed(123)
km.res <- kmeans(df, 4, nstart = 25)
```

As the final result of k-means clustering result is sensitive to the random starting assignments, we specify *nstart = 25*. This means that R will try 25 different random starting assignments and then select the best results corresponding to the one with the lowest within cluster variation. The default value of *nstart* in R is one. But, it's strongly recommended to compute *k-means clustering* with a large value of *nstart* such as 25 or 50, in order to have a more stable result.

Print the results

```
print(km.res)
```

```
## K-means clustering with 4 clusters of sizes 8, 13, 16, 13
##
```

```
## Cluster means:
##       Murder    Assault   UrbanPop        Rape
## 1  1.4118898  0.8743346 -0.8145211  0.01927104
## 2 -0.9615407 -1.1066010 -0.9301069 -0.96676331
## 3 -0.4894375 -0.3826001  0.5758298 -0.26165379
## 4  0.6950701  1.0394414  0.7226370  1.27693964
##
## Clustering vector:
##        Alabama         Alaska        Arizona       Arkansas     California
##              1              4              4              1              4
##       Colorado    Connecticut       Delaware        Florida        Georgia
##              4              3              3              4              1
##         Hawaii          Idaho       Illinois        Indiana           Iowa
##              3              2              4              3              2
##         Kansas       Kentucky      Louisiana          Maine       Maryland
##              3              2              1              2              4
##  Massachusetts       Michigan      Minnesota    Mississippi       Missouri
##              3              4              2              1              4
##        Montana       Nebraska         Nevada  New Hampshire     New Jersey
##              2              2              4              2              3
##     New Mexico       New York North Carolina   North Dakota           Ohio
##              4              4              1              2              3
##       Oklahoma         Oregon   Pennsylvania   Rhode Island South Carolina
##              3              3              3              3              1
##   South Dakota      Tennessee          Texas           Utah        Vermont
##              2              1              4              3              2
##       Virginia     Washington  West Virginia      Wisconsin        Wyoming
##              3              3              2              2              3
##
## Within cluster sum of squares by cluster:
## [1]  8.316061 11.952463 16.212213 19.922437
##  (between_SS / total_SS =  71.2 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

What do we see in the output?

It's possible to compute the mean of each variables by clusters using the original data:

```
aggregate(USArrests, by=list(cluster=km.res$cluster), mean)
```

```
##   cluster   Murder   Assault UrbanPop     Rape
## 1       1 13.93750 243.62500 53.75000 21.41250
## 2       2  3.60000  78.53846 52.07692 12.17692
## 3       3  5.65625 138.87500 73.87500 18.78125
## 4       4 10.81538 257.38462 76.00000 33.19231
```

If you want to add the point classifications to the original data, use this:

```
dd <- cbind(USArrests, cluster = km.res$cluster)
head(dd)
```

```
##            Murder Assault UrbanPop Rape cluster
## Alabama      13.2     236       58 21.2       1
```

```
## Alaska       10.0     263       48 44.5       4
## Arizona       8.1     294       80 31.0       4
## Arkansas      8.8     190       50 19.5       1
## California    9.0     276       91 40.6       4
## Colorado      7.9     204       78 38.7       4
```

Accessing results of kmeans() function

Cluster number for each of the observations

```
#km.res$cluster

head(km.res$cluster, 4)
```

```
##  Alabama   Alaska  Arizona Arkansas
##        1        4        4        1
```

Cluster size

```
km.res$size
```

```
## [1]  8 13 16 13
```

Cluster means

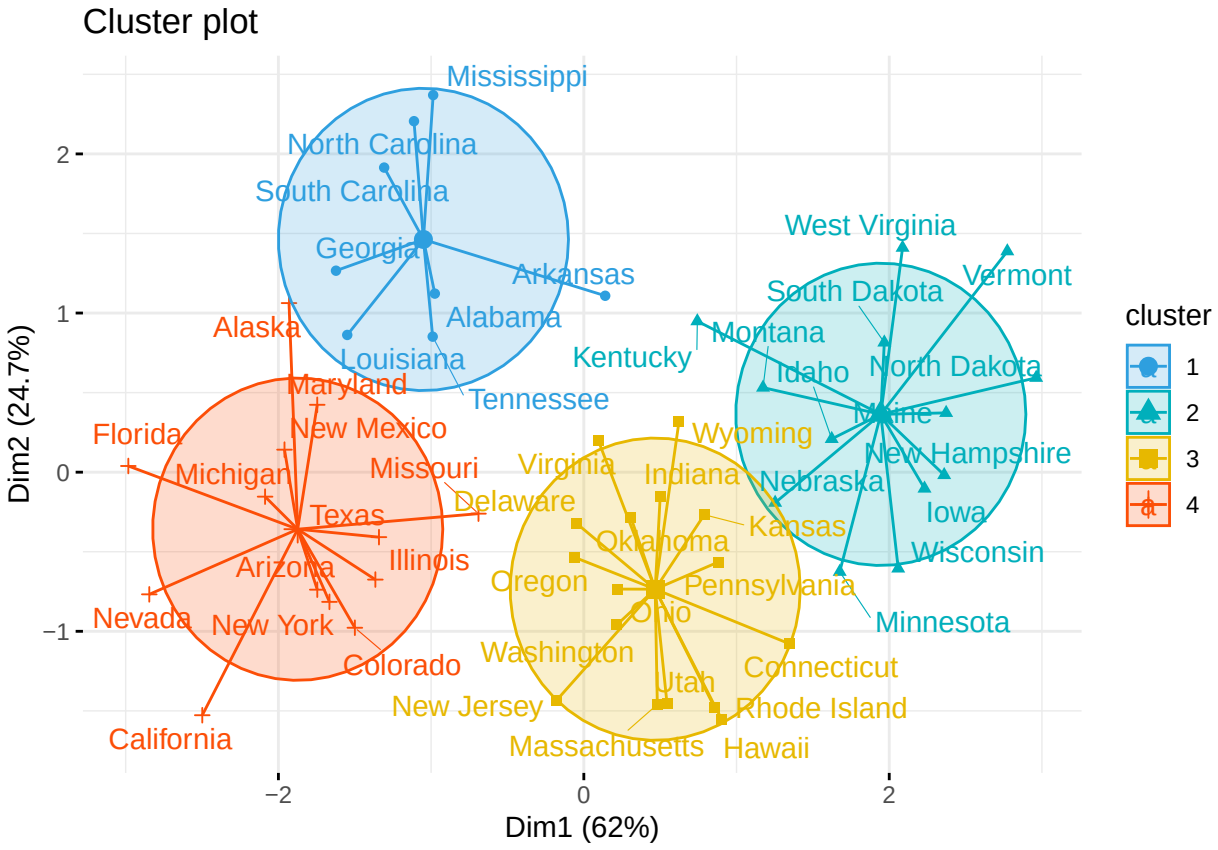
```
km.res$centers
```

```
##       Murder    Assault   UrbanPop        Rape
## 1  1.4118898  0.8743346 -0.8145211  0.01927104
## 2 -0.9615407 -1.1066010 -0.9301069 -0.96676331
## 3 -0.4894375 -0.3826001  0.5758298 -0.26165379
## 4  0.6950701  1.0394414  0.7226370  1.27693964
```

Visualizing k-means clusters

If we have a multi-dimensional data set, a solution is to perform Principal Component Analysis (PCA) and to plot data points according to the first two principal components coordinates.

```
fviz_cluster(km.res, data = df,
             palette = c("#2E9FDF", "#00AFBB", "#E7B800", "#FC4E07"),
             ellipse.type = "euclid", # Concentration ellipse
             star.plot = TRUE, # Add segments from centroids to items
             repel = TRUE, # Avoid label overplotting (slow)
             ggtheme = theme_minimal()
             )
```


Cluster plot
Mississippi
North Carolina
South Carolina
Georgia
Arkansas
Alabama
Louisiana
Tennessee
Alaska
Maryland
New Mexico
Florida
Michigan
Missouri
Delaware
Texas
Illinois
Arizona
Nevada
New York
Colorado
California
West Virginia
Vermont
South Dakota
Montana
Idaho
North Dakota
Maine
New Hampshire
Iowa
Wisconsin
Minnesota
Kentucky
Wyoming
Virginia
Indiana
Nebraska
Kansas
Oklahoma
Oregon
Pennsylvania
Ohio
Washington
Connecticut
Utah
New Jersey
Rhode Island
Massachusetts
Hawaii
Dim2 (24.7%)
Dim1 (62%)
2
1
0
−1
−2
0
2
cluster
1
2
3
4